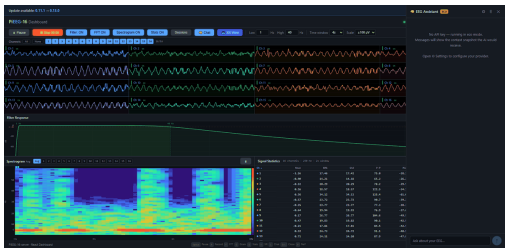
BLUETOOTH INTERFACE & SOFTWARE GUIDE · SKU PIA24023P

BLE Protocol, SDKs & Firmware

Everything needed to read IronBCI-8 from your own code: the BLE service, the 24-byte sample, conversion to microvolts, PiEEG Cloud, PiEEG Server, the Python and JavaScript SDKs, the Android app, open data, and how to update the firmware.



BLE 5	24 B	250/s	0.268	EAREEG
Notify, no pairing	Per sample	Samples per second	µV per LSB	Advertised name

01 SOFTWARE OPTIONS

Option	Best for	Runs on	Start
PiEEG Cloud	Fastest start, dashboards, 3D maps, AI copilot	Chrome / Edge	cloud.pieeg.com
Android app	Mobile demos, recording on the go	Android	mobile_sdk.apk
PiEEG Server	Recording, LSL, OSC, WebSocket to any language	Windows, macOS, Linux	`--device ironbci8`
Python SDK	Learning the BLE protocol, custom processing	Windows, macOS, Linux	`pip install bleak`
JavaScript SDK	Your own web app	Chrome / Edge	`pieeg.js`

02 BLE INTERFACE

Advertised name	`EAREEG` (some units `IronBCI` or `IRONBCI`)
Service UUID	`0000fe42-8e22-4541-9d4c-21edae82ed19`
Data characteristic	`0000fe42-8e22-4541-9d4c-21edae82ed19` — notify
Pairing	Not needed. Connect directly and subscribe to notifications.
Rate	250 samples per second, 8 channels
Streaming	Starts when a client enables notifications; the BLE LED turns on

03 SAMPLE FORMAT

CH1	CH2	CH3	CH4	CH5	CH6	CH7	CH8
bytes 1–3	bytes 4–6	bytes 7–9	bytes 10–12	bytes 13–15	bytes 16–18	bytes 19–21	bytes 22–24

Figure 1 — One sample = 24 bytes. Each channel is 24-bit big-endian two's complement. A notification can carry one or more samples back to back: read it in 24-byte steps.

For research and education only. Not a medical device. Specifications subject to change.

$\mu V = raw \times 4.5 / 16\,777\,215 \times 10^6$

One count ≈ **0.268 µV**. This is the conversion used by the IronBCI SDK and PiEEG Server.

> **NOTE Large DC offset is normal.**
> Raw values carry thousands of µV of electrode offset. Band-pass (e.g. 1–40 Hz) before looking at EEG.

```python
def to_uv(b0, b1, b2):
    raw = (b0 << 16) | (b1 << 8) | b2
    if raw & 0x800000:          # negative value
        raw -= 16777214
    return raw * 4.5 / 16777215 * 1e6

def on_data(_, data):
    for s in range(len(data) // 24):
        smp = data[s*24:(s+1)*24]
        print([to_uv(*smp[3*i:3*i+3]) for i in range(8)])
```

04 PIEEG CLOUD AND ANDROID APP

PiEEG Cloud (browser)

1 Open
cloud.pieeg.com in Chrome or Edge (desktop or Android).

2 Choose
Session Lobby → PiEEG Devices → IronBCI.

3 Connect
Pick the device in the browser's Bluetooth window.

Live waveforms, spectra, 3D topographic maps, recording and playback; processing runs in the browser. Free with PiEEG hardware.

Android app (MIT, Flutter)

1 Install
mobile_sdk.apk from github.com/pieeg-club/ironbci/mobile_app.

2 Set up
Settings → **8 ch**, then connect.

3 Record
Save to start, Stop to finish; share the file.

Flutter source: EEG-Flutter-App-master.zip — build your own app on top of it.

05 PIEEG SERVER

```shell
pip install "pieeg-server[ironbci]"                          # Python 3.10+, installs bleak
pieeg-server --device ironbci8                               # scan, connect, dashboard
pieeg-server --device ironbci8 --ble-name IronBCI            # other advertised name
pieeg-server --device ironbci8 --ble-address AA:BB:CC:DD:EE:FF # skip the scan
pieeg-server --device ironbci8 --filter                      # 1-40 Hz band-pass
pieeg-server --device ironbci8 --record session.csv          # record while streaming
pieeg-server --device ironbci8 --lsl                         # Lab Streaming Layer outlet
pieeg-server --device ironbci8 --osc                         # VRChat OSC bridge
pieeg-server monitor --device ironbci8                       # live view in the terminal
```

Address	What it is
`http://localhost:1617`	Browser dashboard
`ws://localhost:1616`	WebSocket stream: JSON frames with 8 channels in µV

06 PYTHON BLE SDK

Script (SDK/BLE_General)	What it does
`1.Find_Device.py`	Scan and print the device's Bluetooth address
`2.Read_data.py`	Connect and print values in µV
`3.Read_data_graph.py`	Live graph of all 8 channels
`4.Read_gpaph_Autoscale.py`	Live graph with automatic scaling

For research and education only. Not a medical device. Specifications subject to change.

MINIMAL EXAMPLE — PIP INSTALL BLEAK

```
import asyncio
from bleak import BleakScanner, BleakClient

NAME = "EAREEG"                                  # or "IronBCI"
CHAR = "0000fe42-8e22-4541-9d4c-21edae82ed19"

async def main():
    dev = next(d for d in await BleakScanner.discover() if d.name == NAME)
    async with BleakClient(dev.address) as client:
        await client.start_notify(CHAR, on_data)  # on_data from section 03
        await asyncio.sleep(10)                    # stream for 10 s

asyncio.run(main())
```

07 JAVASCRIPT SDK

PIEEG.JS — CHROME / EDGE

```
const eeg = new PiEEG();                          // 'ironbci-8' is the default board
eeg.onBandPowers((p) => console.log('Alpha', p.Alpha));
button.addEventListener('click', async () => {   // Web Bluetooth needs a user click
  await eeg.connectBLE();
});
```

08 OPEN DATA AND PROJECTS

Item	Content
`dataset/eeg_raw_Data_alpha_test.xlsx`	Alpha test, raw µV: 5 s eyes open / 5 s closed, 4 channels, ≈ 50,000 samples (200 s), dry electrodes
`dataset/eeg_band_pass_filtered_alpha_test.xlsx`	Same data, 8–12 Hz band-pass
`Scripts_for_work_with_dataset/`	Band-pass filter and plot your own recordings
`Chat_GPT/Chat_GPT_Real_time.py`	Alpha power every second, labelled by an OpenAI model (learning example)
3D headset	ironbci_3D_EEG_Printable_Headset repository — open printable EEG headset

09 FIRMWARE UPDATE

IronBCI-8 ships programmed. To reload or update the firmware you need an **ST-Link V2** and **STM32CubeProgrammer** (free from ST).

1 Connect
ST-Link to SWD header J2.1 on the BLE board: RST, SWDIO, GND, SWCLK (IB8-PIN-004). Power the device from its battery, Switch 1 ON.

2 Check FUS
In STM32CubeProgrammer connect over SWD, open **Firmware Upgrade Services**, press **Start FUS** and **Read FUS info**.

3 Load the BLE stack
Firmware Upgrade: `stm32wb5x_BLE_Stack_full_fw.bin` at start address **0x080CE000**, then **Start Wireless Stack**.

4 Load the application
Erasing & Programming: the IronBCI application `.hex` from the repository (e.g. `ironbci_15_09_2025.hex`), then disconnect and power-cycle.

5 Verify
The device advertises as EAREEG and streams in PiEEG Cloud or the Android app.

CAUTION Check the stack address.

The correct address depends on the BLE stack version — take it from ST's release notes for the stack you flash (older IronBCI notes list 0x080EC000). A wrong address can block the radio core until FUS is used to delete the stack.

10 RELATED DOCUMENTS

IB8-DS-001 Product Datasheet	**IB8-MD-002** Mechanical & Assembly Drawing	**IB8-PCB-003** PCB & Manufacturing Specification
IB8-PIN-004 Connector, Electrode & Battery Guide	**IB8-QSG-005** Quick Start & Safety Guide	

For research and education only. Not a medical device. Specifications subject to change.